Estudo Teórico de Programação Orientada a Objetos (Java)

Este material foi elaborado com base nos exemplos e exercícios presentes no repositório, especialmente nos pacotes `semana01` a `semana09` e nas listas de exercícios. O objetivo é resumir os principais conceitos da disciplina de POO de forma objetiva e didática.

---

1. O que é Programação Orientada a Objetos?

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento que organiza o software em torno de objetos, que representam entidades do mundo real ou do problema a ser resolvido. Em vez de escrever um programa apenas como sequência de instruções, a ideia é modelar o sistema em classes e objetos.

Os pilares principais da POO são:

- Abstração
- Encapsulamento
- Herança
- Polimorfismo

Além disso, a linguagem Java — usada neste repositório — oferece recursos muito importantes, como:

- Classes e objetos
- Atributos e métodos
- Construtores
- Métodos acessores (`get` e `set`)

- Interfaces
- Exceções
- Herança e classes abstratas

---

2. Classe e objeto

Classe

Uma classe é um molde ou projeto. Ela descreve as características e comportamentos de um tipo de entidade.

Exemplo no repositório: [poo/src/semana04/Calculadora.java]

A classe `Calculadora` define operações matemáticas como:

- `somar(int a, int b)`
- `subtrair(int a, int b)`
- `multiplicar(int a, int b)`
- `dividir(int a, int b)`
- `raiz(int a)`

Esses métodos representam ações que a calculadora pode executar.

Objeto

Um objeto é uma instância concreta de uma classe. Ou seja, é a materialização do molde em memória.

Exemplo de criação de objeto:

```
Calculadora c = new Calculadora();
int resultado = c.somar(3, 5);
```

Mesmo que a classe seja apenas um projeto, o objeto é aquilo que realmente participa da execução do programa.

---

3. Atributos e métodos

Atributos

Atributos são as propriedades de uma classe. Eles representam dados ou estados.

Exemplo em poo/src/semana07/Animal.java:

```
private int idade;
private String nome;
```

Esses atributos descrevem o estado de um animal.

Métodos

Métodos representam comportamentos: ações que o objeto pode realizar.

Exemplo em poo/src/semana07/Animal.java:

```
public void comer() {
    System.out.println("Animal comendo...");
}
```

Esses conceitos fazem parte da base da POO: dados e ações estão unidos dentro da classe.

4. Encapsulamento

Encapsulamento é o princípio de esconder detalhes internos da implementação e expor apenas o necessário para o restante do programa.

No repositório, esse conceito aparece claramente em poo/src/semana07/Animal.java e em poo/src/atividadeExercicios/EncapLampada.java.

Características do encapsulamento

- Atributos privados ou protegidos
- Uso de métodos públicos para acesso e alteração
- Controle de validação dos dados

No exemplo da classe `Animal`:

```
private int idade;

public int getIdade() {
    return idade;
}

public void setIdade(int idade) {
    if(idade > IDADE_MINIMA && idade < IDADE_MAXIMA) {
        this.idade = idade;
    }
}
```

Esse código mostra que a classe controla a forma como a idade pode ser alterada. Isso evita estados inválidos.

Vantagens

- Segurança dos dados
- Facilita manutenção
- Reduz erros por manipulação indevida de atributos
- Aumenta coesão da classe

---

5. Modificadores de acesso

Java possui modificadores de acesso que controlam a visibilidade de atributos, métodos e classes.

`public`

Permite acesso por qualquer classe.

`private`

Restringe o acesso ao interior da própria classe.

`protected`

Permite acesso dentro do mesmo pacote e também em subclasses.

`package` (default)

Disponível apenas dentro do mesmo pacote, sem palavra-chave explícita.

No código de poo/src/semana07/Animal.java, há comentários que explicam essa ideia:

```
protected int id;
private int idade;
```

Isso mostra que a classe usa controle de visibilidade para proteger e organizar a implementação.

---

6. Construtores

Construtor é um método especial usado para inicializar um objeto quando ele é criado.

Em poo/src/semana07/Animal.java:

```
public Animal() {
}

public Animal(int id, int idade, String nome) {
    this.id = id;
    setIdade(idade);
    this.nome = nome;
}
```

O construtor define como um objeto será criado. Ele pode receber parâmetros, como `id`, `idade` e `nome`, garantindo que o objeto comece em um estado válido.

---

7. Herança

Herança permite que uma classe herde atributos e métodos de outra classe. Isso ajuda a reutilizar código e modelar relacionamentos entre tipos.

Exemplo no repositório:

- poo/src/semana07/Animal.java
- poo/src/semana07/Gato.java
- poo/src/semana07/Elefante.java
- poo/src/semana07/Pessoa.java
- poo/src/semana07/PessoaFisica.java

Exemplo conceitual

```
public class Gato extends Animal {
    // atributos e métodos específicos de Gato
}
```

A classe `Gato` herda características de `Animal`, como nome e idade, e pode acrescentar seus próprios comportamentos.

Vantagens da herança

- Reuso de código
- Melhor organização da hierarquia
- Modelagem mais fiel do mundo real

Atenção

Herança deve ser usada com cuidado, principalmente quando a relação entre classes é realmente de generalização/especialização.

---

8. Classes abstratas

Uma classe abstrata não pode ser instanciada diretamente. Ela serve como base para outras classes.

Exemplo em poo/src/semana07/Animal.java:

```
public abstract class Animal {
```

A classe `Animal` define comportamento comum, mas não representa um objeto concreto em si. Isso faz sentido porque um animal genérico não é uma instância específica.

Quando usar?

Quando há uma classe que:

- representa uma ideia geral;
- possui métodos que serão implementados por subclasses;
- precisa funcionar como base de herança.

---

9. Interfaces

Interface é um contrato. Ela define métodos que as classes que a implementam devem fornecer.

Exemplo no repositório em:

- poo/src/semana09/Motor.java
- poo/src/semana09/Capota.java
- poo/src/semana09/Automovel.java

A classe `Automovel` implementa duas interfaces:

```
public class Automovel implements Motor, Capota {
```

Isso significa que a classe precisa implementar os métodos definidos por essas interfaces, como:

- `ligar()`
- `desligar()`
- `acelerar()`
- `abrir()`
- `fechar()`

Diferença entre classe abstrata e interface

- Classe abstrata pode ter atributos e métodos concretos
- Interface define apenas contrato, sem implementação concreta

- Uma classe pode implementar várias interfaces, mas herdar de apenas uma classe

---

10. Polimorfismo

Polimorfismo significa que objetos de diferentes classes podem ser tratados da mesma forma quando compartilham uma interface ou uma classe pai.

Em Java, isso acontece principalmente por:

- sobreposição de métodos (`@Override`)
- uso de classes abstratas
- uso de interfaces

Exemplo prático: um `Automovel`, um `Aviao` e um `Navio` podem implementar a interface `Motor` e, portanto, compartilhar o comportamento de ligar, desligar e acelerar.

Resultado

O código pode tratar diferentes objetos como se todos fossem do mesmo tipo, sem precisar conhecer o detalhe de cada implementação.

---

11. Associações e composição

Além das relações de herança, os objetos podem se relacionar de outras formas.

Exemplo em [poo/src/semana08/Funcionario.java]:

```
public class Funcionario {
    public int id;
```

```
    public String nome;
    public Departamento departamento;
}
```

Aqui, um funcionário está associado a um departamento. Essa relação mostra que um objeto pode ter referência a outro objeto.

Importância

- Modela dependências entre entidades
- Representa o mundo real de forma mais fiel
- Permite organizar sistemas maiores

---

12. Tratamento de exceções

Exceções são eventos que acontecem durante a execução e que interrompem o fluxo normal do programa, como divisão por zero ou entrada inválida.

Exemplo em poo/src/semana03/Excecao01.java:

```
try {
    int calculo = 1000 / a;
}
catch(NumberFormatException erro) {
    // trata entrada inválida
}
catch(ArithmeticException erro) {
    // trata divisão por zero
}
finally {
    // executa mesmo com erro
}
```

Estrutura básica

- `try`: bloco que pode gerar erro
- `catch`: trata a exceção
- `finally`: executa no final, independentemente do sucesso ou falha

Por que isso é importante?

Porque programas reais precisam reagir de forma segura a entradas do usuário, recursos ausentes, operações inválidas e falhas de sistema.

---

13. Métodos estáticos

Métodos estáticos pertencem à classe e não a um objeto específico.

Exemplo em poo/src/semana04/Calculadora.java:

```
public static int somar(int a, int b) {
    return a + b;
}
```

Nesse caso, o método pode ser chamado sem instanciar um objeto:

```
int total = Calculadora.somar(2, 3);
```

Uso comum

- operações utilitárias
- classes de apoio
- funções matemáticas

---

14. Encapsulamento em exercícios práticos

O material do repositório inclui exemplos de encapsulamento com lâmpada, estado e controle de ações:

- poo/src/atividadeExercicios/EncapLampada.java
- poo/src/atividadeExercicios/EncapEstadoLampada.java
- poo/src/atividadeExercicios/EncapGerenciarLampada.java

A ideia é que o objeto controle o seu próprio estado e expose apenas ações relevantes, como:

- `acender()`
- `apagar()`
- `ajustarMeiaLuz()`

Esse padrão é um dos melhores exemplos de encapsulamento em aplicação real.

---

15. Resumo dos principais conceitos da disciplina

A disciplina de POO pode ser sintetizada em alguns pontos fundamentais:

1. Classes definem modelos de objetos.
2. Objetos são instâncias de classes.
3. Atributos armazenam dados do objeto.
4. Métodos representam comportamentos.
5. Encapsulamento protege dados e controla acesso.
6. Herança reutiliza código.
7. Abstração reduz detalhes irrelevantes.
8. Interfaces definem contratos.
9. Polimorfismo permite tratar diferentes objetos de forma uniforme.
10. Exceções tornam o programa mais robusto.

---

16. Conclusão

O repositório demonstra claramente que a POO não é apenas teoria abstrata: ela está diretamente ligada ao desenvolvimento de software mais organizado, reutilizável e fácil de manter. A partir dos exemplos de cálculos, classes de domínio, encapsulamento, herança e tratamento de erros, é possível observar como a abordagem orientada a objetos ajuda a modelar problemas reais de maneira mais clara.

Em outras palavras, a POO permite transformar ideias em estruturas organizadas, onde cada objeto tem responsabilidade, dados e comportamento próprios.

---

17. Sugestões de estudo

- Releia os exemplos de poo/src/semana04
- Estude as classes de poo/src/semana07 e poo/src/semana09
- Compare encapsulamento com herança e interfaces
- Pratique criando pequenas classes novas para modelar objetos do cotidiano
- Faça exercícios de implementação usando atributos privados e métodos públicos

Esse processo de estudo e prática é o caminho mais eficiente para fixar os conceitos de POO.